Programming in C

Raphael 'kena' Poss

This syllabus covers the part "Programming in C" of the course "Datastructuren" delivered to 1st year BSc INF and 2nd year BSc KI at the University of Amsterdam.

Contents

1 Course introduction

- Staff:
  - Teachers: Tim Doolan + Raphael Poss
  - Assistants: Jetske Beks, Marco Brohet, Lorian Coltof, Ysbrand Galama, Devin Hillenius, Erik Kooistra, Kyrian Maat, Thomas Schaper, David Veenstra, Youri Voet, Simon Polstra
- Course page on Datanose:
  - INF: https://datanose.nl/#course[56578]
  - KI: https://datanose.nl/#course[48396]
- **Only 1 Blackboard page**
- Lectures:
  - "Datastructuren" every Thursday 3-5pm SP C0.05
  - "Programming in C" - 6 lectures in February - irregular timetable
  - Check Datanose!
- Study work:
  - Labs - 3 groups, 2×2 hours / group / week
  - One assignment per week, posted on Blackboard
  - ~10 hours self study / week
  - Check Datanose! check BlackBoard!
- Grading:
  - INF: 15% PAV, 55% labs, 30% exam
  - KI: 2/3 labs, 1/3 exam
  - Check "Studiewijzer" on Blackboard
- Contact: questions / requests etc
  - Personal stuff: study advisor
  - General stuff: ds2017-assist@list.uva.nl
  - Discussion: IRC channel? Slack?

poll: progc1 poll link - results link

- Getting started: you'll need 1) a C compiler 2) a prompt to run your own programs.

  Now! Otherwise: http://codepad.org/

2 Introduction to C by example

An example program:

```
main() { return 42; }
```

What does this do? Try it!

Steps:

1. Put code into **text file**, name file with extension **.c**
   - Pick your editor: `gedit`, `mousepad`, `gvim`, `vim`, `emacs`, etc.
   - Attention: must be "plain text" - no Word, OpenOffice, etc.
2. Compile code, e.g. with command "`cc`"

    - By default this produces a file named `a.out` - rename with `-o` if needed
3. Run program
4. Ask shell for exit status: `echo $?`

Demo:

```
$ gedit test1.c
$ cc test1.c     # produces "a.out"!
$ ./a.out
$ echo $?
42
$ cc test1.c -o test1
$ ./test1
$ echo $?
42
```

Let's try another:

```
main() { return 1 + 2 * 3; }
```

What does this program return?

```
$ gedit test2.c  # place code in file
$ cc test2.c
$ ./a.out
$ echo $?
(your guess here)
```

Attention: C uses **operator precedence** to determine evaluation order. Like in maths.

Another one (`test3.c`):

```
main
(
)
{
return
40
+
2
;
}
```

What does this tell you?

The C language is (mostly) insensitive to whitespace.

Let's look at the output of the C compiler a bit closer:

```
$ cc test2.c
test2.c:1:1: warning: return type defaults to ‘int’ [-Wimplicit-int]
 main() { return 42; }
 ^~~~
```

What's going on?

The C language is very simple, **you must always tell it the type of things.**

Our `main` function returns an integer number, so we need to explain this (`test4.c`):

```
// Before: "int" was omitted.
// Now: we place "int" before "main".
int main() { return 42; }
```

Try it! The C compiler should not complain any more.

Oh, and what did we do here? Those two lines:

```
// Before: "int" was omitted.
// Now: we place "int" before "main".
```

The C code can contain **comments**.

A comment in C is enclosed either by `//` ... [newline], or `/*` ... `*/`.

Use comments to explain your code! Always.

2.1 Summary

- The smallest program in C starts with "`int main() {`" and ends with "`}`"
- **Expressions** order operations using precedence, like in math.
- A program must **return a value** to the environment with "`return`".
- Whitespace does not matter.
- Comments are ignored by the C language, but matter to the human reader.

3 How to speak C aloud

You must learn how to read C code aloud to someone else, so they can write what you say. You must also learn to listen to someone speaking C to you and write down what they say.

For this you need words:

C thing	English	Nederlands
{ }	brace	accolade
( )	parenthesis	parenthese, rond haakje
;	semicolon	puntkomma
	space	spatie
	newline	nieuw regel

Don't forget to say “open” and “close” (“gesloten”) when needed.

Try it:

```
int main() {
  return 42;
}
```

You will need some more words to read most simple C programs:

C thing	English	Nederlands
#	hash, pound	hekje
< >	lower than, greater than	kleiner dan, groter dan
"	double quote	dubbel aanhalingsteken
\	backslash	backslash
'	single quote	enkel aanhalingsteken, apostrof
.	dot, period	punt
,	comma	komma
[ ]	bracket	haakje, vierkant haakje
/	slash	slash, schuine streep
-	dash, minus	streepje, minteken
_	underscore	underscore, laag streepje
|	pipe, vertical bar	pipe, sluisteken
*	star	sterretje
&	ampersand, “and”	ampersand, “en”
->	arrow, “min greater than”	pijltje, “streepje groter dan”
!=	different from	anders dan, ongelijk aan

With this you can read e.g. the following aloud:

```
#include <stdio.h>
int main() {
  printf("hello, world.\n");
  return 0;
}
```

The remaining words are somewhat less common but also needed later, so learn them:

C thing	English	Nederlands
^	hat	dakje
:	colon	dubbelepunt
!	bang, exclamation mark	uitroepteken, oproepteken
?	question mark	vraagteken

3.1 Summary

- C uses a lot of punctuation - you'll learn all of it incrementally.
- Most punctuation signs are called the same as when you study literature.
- Different words for different grouping: **braces, brackets, parentheses** (accolades, vierkante haakjes, parentheses).
- Special cases:
  - "*" is “star” (“sterretje”), not “asterisk”
  - "|" is “pipe”
  - "->" is called “arrow” / “pijltje”
  - "!=" is called “different from” / “anders dan”
  - "&" is more often called “and” (“en”) than “ampersand”

4 Printing stuff

Consider the example from above:

```
#include <stdio.h>
int main() {
  printf("hello, world.\n");
  return 0;
}
```

Try it! What does it do?

Another one:

```
#include <stdio.h>
int main() {
  printf("number: %d\n", 40 / 2);
  return 0;
}
```

Try it! What does it do?

4.1 Intro to `printf`

`printf` is defined like this:

```
int printf(string fmt, ...);
```

Which means it takes a string as first argument, then a variable number of arbitrary arguments.

What it does: use the first “format” argument as **template** to print the rest of the arguments.

Example use	Example output
`printf("hello");`	`hello`
`printf("hel %d lo", 123);`	`hel 123 lo`
`printf("%s, hello", "jane");`	`jane, hello`
`printf("hello %s", "jane");`	`hello jane`

Note how `%d` and `%s` are used:

- `%d` for **integer numbers**
- `%s` for **strings**

Summary:

- `printf` takes always at least one argument. This first argument must be a string.
- For every `%` in the format, `printf` takes the next remaining argument and prints it.

- Use `%s` for strings, `%d` for numbers.
- If you want a newline at the end of the output, you need to tell `printf` explicitly with "`\n`".

4.2 Intro to numbers and characters

What does this program do?

```
int main() {
  printf("hello %d\n", 'i');
}
```

Try it!

poll: progc3 poll link poll results

What's going on?

- The special syntax `'i'` in the program is translated to the numeric value of the character "i". See the ASCII table for a correspondance.
- The function `printf` prints this number with `%d` as usual.

What does this program do?

```
int main() {
  printf("hello %c\n", 105);
}
```

Try it!

```
$ gedit test5.c
$ cc test5.c
$ ./a.out
hello i
```

What's going on?

The format code `%c` **prints a character with the code given** in the next number argument.

The following three programs print **the same thing**:

```
int main() {
  printf("hello %c\n", 105);
}
```

```
int main() {
  int  mychar = 'i';
  printf("hello %c\n", mychar);
}
```

```
int main() {
  printf("hello %c\n", 'i');
}
```

4.3 Intro to `putchar`

The function `putchar` is defined as follows:

```
// Print the character with code c.
int putchar(int c);
```

Another example program:

```
#include <stdio.h>
int main() {
  putchar('h');
  putchar('i');
  putchar('!');
  putchar('\n');
  return 0;
}
```

What do you think?

Now something slightly more complicated:

```
#include <stdio.h>
int main() {
  putchar('h');
  putchar(105);
  return 0;
}
```

What output do you see from this program when you run it from the command line?

poll: progc4 poll link - poll results

What's going on: the shell is confused when the program does not terminate its output with a newline character.

Do not forget the final newline character.

4.4 Summary

- `printf(fmt, ...)` to print stuff with a format (template).
  - `%d` for numbers,
  - `%s` for strings,
  - `%c` for characters from a code.
- `putchar(c)` to print individual characters from their code.
- The syntax `'X'` (with single quotes!) translates the character "X" to a number automatically.
- Don't forget to print a newline character (`\n`) at the end of output.
- Use "`#include <stdio.h>`" at the beginning if you want to use `printf` or `putchar` without warnings.

5 Interacting with the environment

You already know how to *output* stuff from a program:

- using the return value;
- using `printf` to output text.

Now, how does input work?

There are many mechanisms for input, we will use just two of them:

- **command-line arguments**
- **standard input**

5.1 Command-line arguments: `argc` and `argv`

To accept command-line arguments your program must start like this:

```
// New: argc, argv
int main(int argc, string argv[]) {
...
}
```

Let's try it out:

```
#include <stdio.h>
int main(int argc, string argv[]) {
   printf("%d\n", argc);
}
```

Run it (you will need to download this file easy.h):

```
$ gedit test6.c # put code in file
$ cc test6.c -include easy.h
$ ./a.out
1
$ ./a.out a
2
$ ./a.out a a a a a
6
$
```

What's going on?

The special variable **argc** automatically receives the number of arguments, plus one. (Why "plus one"? See below.)

Let's try out some more:

```
#include <stdio.h>
int main(int argc, string argv[]) {
   printf("%s\n", argv[0]);
}
```

```
$ gedit test7.c # put code in file
$ cc test7.c -include easy.h
$ ./a.out
./a.out
$
```

What just happened? Hmmm.

Let's try rename the program file generated by `cc`:

```
$ mv a.out   sometest
$ ./sometest
./sometest

$ mv sometest yahoo
$ ./yahoo
./yahoo
```

What's going on?

The first element of the `argv` array automatically receives the name of the program.

So, `argv` is an "array." What's that?

An array is a sequence of values next to each other in memory.

In C the elements of an array are always numbered starting from 0.

So what is in `argv` in the 2nd position? Let's find out.

```
#include <stdio.h>
int main(int argc, string argv[]) {
   printf("%s\n", argv[1]);
}
```

```
$ gedit test8.c # put code in file
$ cc test8.c -include easy.h
$ ./a.out hello
hello
$ ./a.out world
world
```

So `argv` automatically contains the command-line arguments.

5.1.1 Be careful with arrays

What happens if we try to use an element that does not exist?

```
$ ./a.out # no extra argument!
Segmentation fault (core dumped)
$
```

Woah. What's going on?

Trying to use an array element that does not exist is BAD.

Don't look at invalid array positions!

Don't write a program that accesses array elements that do not exist!

In the example above this is easy to check: the variable `argc` tells how many elements there are.

Try this:

```
#include <stdio.h>
int main(int argc, string argv[]) {
   // Check the number of arguments given.
   if (argc != 2) {
     printf("woops!\n");
     return 1;
   }

   // Do stuff.
   printf("%s\n", argv[1]);
   return 0;
}
```

```
$ gedit test9.c # put code in file
$ cc test9.c -include easy.h
$ ./a.out
woops!
$ ./a.out hello
hello
```

Easy? Here you've used `if ... else ...` to make a choice. We'll see more about that in a moment.

Now, let's *print all the things!*

```
int main(int argc, string argv[]) {
   for (int i = 1; i < argc; i = i + 1) {
      printf("%s\n", argv[i]);
   }
   return 0;
}
```

```
$ gedit test10.c # put code in file
$ cc test10.c -include easy.h
$ ./a.out hello world
```

What does this program print?

poll: progc5 poll link poll results

5.1.2 Accepting numbers from the command-line

`argv` contains strings. What if we want to turn them into numbers to compute with numbers?

Say, we want to make a program that computes the sum of its two arguments.

For this you'll need the following function:

```
// atoi onverts the string argument and produces a number (Ascii To Integer).
int atoi(string v);
```

It's predefined in `stdlib.h`, which you'll need to include. Try this:

```
#include <stdio.h>
int main(int argc, string argv[]) {
   // Check the number of arguments given.
   if (argc != 3) {
     printf("woops!\n");
     return 1;
   }

   // Do stuff.
   return atoi(argv[1]) + atoi(argv[2]);
}
```

```
$ gedit test11.c # put code in file
$ cc test11.c -include easy.h
$ ./a.out 100 23
$ echo $?
123
$
```

You can also try this: (you'll need easy.h)

```
#include <stdio.h>
int main(int argc, string argv[]) {
   // Check the number of arguments given.
   if (argc != 3) {
     printf("woops!\n");
     return 1;
   }

   // Do stuff.
   printf("%d\n", atoi(argv[1]) + atoi(argv[2]));
}
```

```
$ gedit test12.c # put code in file
$ cc test12.c -include easy.h
$ ./a.out 100 23
123
$
```

And maybe you'll find this more readable:

```
#include <stdio.h>
int main(int argc, string argv[]) {
   // Check the number of arguments given.
   if (argc != 3) {
     printf("woops!\n");
     return 1;
   }

   // New: store indermediate results
   // in variables.
   int a = atoi(argv[1]);
   int b = atoi(argv[2]);
   printf("%d\n", a + b);
}
```

5.1.3 Command-line: summary

- use `int main(int argc, string argv[])` to start the program if you want to use command-line arguments.
- `argc` contains the number of arguments plus one (the program name itself).
- `argv` is an array containing the argument strings.
  - starting at position 0 with the program name.
  - last item is at position `argv-1`.
- do not access array elements that do not exist!
- use `atoi` (`#include <stdlib.h>`) to convert a string to a number which you can compute with.

5.2 Standard input

That's the other common way to read input.

It gives access to data that is entered *after the program has already started*.

5.2.1 Reading character by character

You'll use this function:

```
// getchar reads a single character and returns its code.
int getchar();
```

For example:

```
#include <stdio.h>
int main() {
   // Read a character (its code).
   int c = getchar();

   // Print the code.
   printf("hello %d\n", c);
   return 0;
}
```

Run this! What do you see?

```
$ gedit test14.c # put the code in the file
$ cc test14.c
$ ./a.out
(type something here)
```

For example: if you type “i”, your program prints ... 105. If you type “e” your program prints 101.

If you just press the enter key your program prints ... 10, the code for the newline character (see the ASCII table).

Note: the execution of the program is blocked until `getchar()` successfully reads a character. If you do not provide input, your program never terminates.

You can also pass input from another program, for example:

```
$ echo i | ./a.out
hello 105
```

This tells the shell to redirect the output of `echo` to the standard input of your program. This way `echo` provides the character “i” to the standard input, and you do not need to type it in.

5.2.2 Handling “End of File”

Suppose you want to write a program that prints the code of all the characters available on standard input, then stops.

We need a way to know “when there is no more input”.

Conveniently, **getchar() returns the special value `"EOF"` when there is no more input available.**

Try this:

```c
#include <stdio.h>
int main() {
   while(1) {
     // Read a character code.
     int c = getchar();

     // Was this the end of input?
     if (EOF == c) {
       return 0;
     }

     // No, print the character.
     printf("hello %c\n", c);
   }
   return 0;
}
```

```
$ gedit test15.c # put the code in the file
$ cc test15.c
$ echo "world" | ./a.out
hello w
hello o
hello r
hello l
hello d
hello
$
```

What just happened?

- The program contains a loop `while(1) { ... }`
- Inside this loop, the program:
  - reads one character with `getchar()`
  - checks if the End-of-File condition was encountered `EOF == c`
    - if it was encountered, the program terminates with `return`
  - then it prints the character
  - and the loop executes again.

Note: **This general program structure (a loop containing code to read, test, compute, print something repeatedly) is very common. It is called an "event loop" or sometimes "read-eval-print-loop" (REPL).**

5.3 Summary of all the things

- Read from command-line arguments with: `int main(int argc, string argv[])`
  - Be careful with array sizes.
- Read from standard input with `getchar()` and `EOF`.
  - General structure "`while(1) { read ... test ... print }`" is very common.

6 Functions

You've seen plenty of functions already!

`main`, `printf`, `putchar`, `getchar`, `atoi` are all functions.

You can make your own functions too!

6.1 Function definitions

You can make your own functions!

```c
// square returns the value of its argument, squared.
int square(int x) {
  x = x * x;
  return x;
}

int main() {
  int c = square(1);
  printf("%d\n%d\n", c, square(2));
  for (int i = 3; i <= 10; i++)
    printf("%d\n", square(i));
  return 0;
}
```

```
$ gedit test18.c # put the code in the file
$ cc test18.c
$ ./a.out
1
4
9
...
89
100
```

6.2 Arguments passed by value

In general a function receives a *copy* of its arguments: the variables in the caller context are not modified!

Consider the following:

```
int square(int x) {
   x = x * x;
   return x;
}

int main() {
   int x = 2;
   int y = square(x);
   x = x + 1;
   printf("%d %d\n", x, y);
   return 0;
}
```

What does this program print?

poll: progc9 poll link - poll results

What's going on? In `square` the variable `x` is initialized with a *copy* of `x` in `main`.

The variable named "`x`" in `square` and the variable named "`x`" in `main` are two different variables!

6.3 Arguments passed by reference

Sometimes, you really need a function to be able to modify the original variable in its caller.

You can do this as follows:

```
#include <stdio.h>

// this function copies its first argument to the 2nd (writing in
// the caller's context) if it is non-zero, then returns the
// value of the first argument.
int copy_if_nonzero(int x, ref_to(int) y) {
   if (x != 0)
      deref(y) = x;

   return x;
}

int main() {
   int z = 42;

   copy_if_nonzero(0, ref(z));
   printf("%d\n", z);

   copy_if_nonzero(12, ref(z));
   printf("%d\n", z);

   return 0;
}
```

Try with: (you'll need easy.h)

```
$ gedit test20.c # put code in file
$ cc test20.c -include easy.h
$ ./a.out
42
12
```

What's going on?

- "`ref_to(int) y`" declares the parameter `y` to be "a reference to another variable of type `int`"
- the operator `deref(y)` says "access the variable referenced to by `y`"
- the operator `ref(z)` (in `main`) says "create a reference to the variable `z`"

What does the following program print?

```
#include <stdio.h>

int square_inplace(ref_to(int) a) {
   int n = deref(a) * deref(a);
   deref(a) = n;
}

int main() {
   int x = 2;
   square_inplace(ref(x));

   ref_to(int) y = ref(x);
   int z = deref(y);

   square_inplace(ref(x));
   square_inplace(ref(z));

   printf("%d %d %d\n", x, deref(y), z);
   return 0;
}
```

poll: progca poll link poll results

What's going on?

1. `int x = 2` : makes a real variable named `x`, initialized to 2;
2. `square_inplace(ref(x))` : `x` updated to 4;
3. `ref_to(int) y = ref(x)` : `y` becomes another name for the variable already named `x`;
4. `int z = deref(y)` : makes another real variable named `z`, initialized to the current value of the variable named `y` (also named `x`), i.e. 4;
5. `square_inplace(ref(x))` : `x` updated to 16;
6. `square_inplace(ref(z))` : `z` (separate from `x`!) updated to 16;
7. `printf`: current value of `x` is 16; current value of `deref(y)` is the same as `x`, 16 too; current value of `z` is 16.

6.4 Procedures

What if you want to define a function *that has no return value*? For example a function that only prints stuff and does not compute.

This is a special case:

use the `void` return type to define *procedures* (functions without a result).

For example:

```
#include <stdio.h>

// New! "void"
// sayhello greets the person whose name is given as argument.
void sayhello(string name) {
  printf("hello, %s!\n", name);
  // No return value in function with "void" return type.
}

int main(int argc, string argv[]) {
  sayhello("joe"); sayhello("jane");
  if (argc == 2)
    sayhello(argv[1]);
  return 0;
}
```

Try it with: (you'll need easy.h)

```
$ gedit test19.c # put code in file
$ cc test19.c -include easy.h
$ ./a.out
hello joe!
hello jane!
$ ./a.out robert
hello joe!
hello jane!
hello robert!
$
```

6.5 Summary

- you can make your own functions! Syntax: `int foo(...) { ... }`
- return values with `return` — this also stops execution in the function!
- by default, *arguments passed by value*.
- you can *pass arguments by reference*:
  - using `ref_to(T)` to declare a reference parameter,
  - `deref(r)` to access the variable referenced to by `r`, and
  - `ref(x)` to make a new reference to `x`.
  - This syntax needs easy.h!
- make procedures (no return value) by replacing the return type by "`void`"

7 What you just learned without me telling

You can **compute stuff** in C using *expressions*.

You can **decide and do stuff** using *control structures*.

You can **store** stuff using *variables*.

You can **abstract** stuff using *functions*.

You can "access variables remotely" with *references*.

You already know all this from the examples!

Now, let's bring some structure to it.

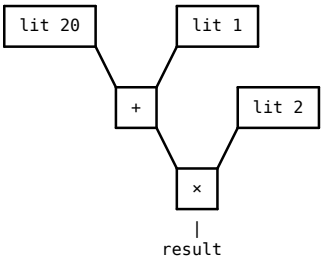
8 Expressions

Expressions in the program are evaluated to a **value** during execution.

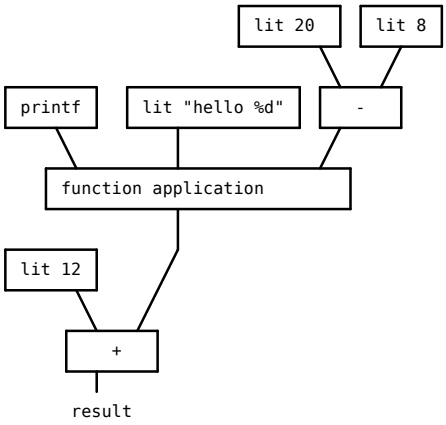
Category	Format	What we call it in source code	Its value during execution
literal	`"hello"`	a string literal	the string
literal	`'X'`	a character literal	the character's code (e.g. `'X'` has value 88)
literal	`123`	a number literal	the number
arithmetic	`a + b`	"`a` plus `b`"	the sum of the values of `a` and `b`
arithmetic	`a * b`	"`a` times `b`"	the product of the values of `a` and `b`
comparison	`a < b`	"`a` lower than `b`"	1 if the value of `a` is smaller than the value of `b`, 0 otherwise
comparison	`a == b`	"`a` test equals to `b`"	1 if the value of `a` is equal the value of `b`, 0 otherwise
comparison	`a != b`	"`a` test different from `b`"	0 if the value of `a` is equal the value of `b`, 1 otherwise
indexing	`a[b]`	"`a` bracket `b` close bracket", or "`a` indexed by `b`"	the `b`-th element of `a`
application	`a(b,c)`	"`a` parenthesis `b` comma `c` close parenthesis" or "function `a` applied to arguments `b` and `c`"	the result returned by function `a` called with the value of `b` and `c` as arguments
misc	`(a)`	"`a` enclosed in parentheses"	the value of `a`
misc	`ref(v)` or `addr_of(v)` (with easy.h)	"reference to `v`" or "address of `v`"	a reference to `v`
misc	`deref(r)` (with easy.h)	"deref `r`" or "`r`, dereferenced"	the variable referenced to by `r`

Example expressions in C:

```
(20 + 1) * 2
```




```
12 + printf("hello %d", 20 - 8)
```



Some more expressions:

```
a < (3 + b)

x[y - 123]
```

To predict the value of an expression, imagine the expression tree and evaluate the tree starting from the leaves.

Quiz: what does the following program print?

```
int main() {
   int c = "hello"['f'-'e'];
   printf("%d\n", c - 1);
   return 0
}
```

poll: progc6 poll link - poll results

8.1 Summary

- There are many sorts of expressions in C: `a + b`, `a * b`, etc.
- A function call is an expression too: `a(b)`
- Simple values, also called *literals*, are expression too: `'a'`, `"hello"`, `123`
- During execution the program *evaluates* each expression to determine its *value*.
  - Starting at the leaves of the expression tree.

9 Control structures

9.1 Conditional statement (`if`) and simple loops (`while`)

These you need to remember:

Syntax	How we call this
`if ( ... ) ...`	an `if` statement
`if ( ... ) ... else ...`	an `if`..`else` statement
`while ( ... ) ...`	a `while` loop
`do ... while ( ... )`	a `do`..`while` loop

Their behavior should be transparent if you already know stuff from other programming languages. Otherwise:

- `if (a) b`

  evaluates expression `a`, then runs `b` if the value of `a` is non-zero.

For example:

```
if (argc != 2) {
  printf("woops!\n");
  return 0;
}
```

- `if (a) b else c`

  evaluates expression `a`, then runs `b` if the value of `a` is non-zero, otherwise runs `c`.

  For example:

```
if (argc == 2)
  return atoi(argv[1]);
else
  return 42;
```

- `while (a) b`

  evaluates expression `a`, then runs `b` if the value of `a` is non-zero, then repeats.

  For example:

```
while(1) { int c = getchar(); ... }
```

- `do b while(a)`

  runs `b` once. Then runs `while(a) b`.

9.2 Braces or no braces?

This code is valid:

```
if (argc == 2)
  return atoi(argv[1]);
else
  return 42;
```

This code is valid, too:

```
if (argc == 2) {
  return atoi(argv[1]);
} else {
  return 42;
}
```

What does the following print?

```
if (4 == 2 + 1)
   printf("this is ");
   printf("false\n");
```

General rule: **the braces group statements together.**

Also: **Beware! `if..else` binds tightly.**

What does the following program do?

```
#include <stdio.h>
int main() {
   if (3 == 2 + 1)
      if (4 == 2 + 1)
        printf("a");

   else
     printf("b");

   printf("\n");
   return 0;
}
```

poll: progc7 poll link - poll results

What's going on?

`else` binds to the closest `if`:

```
    if (a) if (b) c else d
```

is equivalent to

```
    if (a) { if (b) c else d }
```

always, irrespective of spacing!

When in doubt, use braces.

Actually, **use braces always when you're a beginner.** That will prevent mistakes that are hard to find.

9.3 `for` loops

Remember this:

- **Usually, you can understand `for` loops easily using a common pattern.**
- **When you can't recognize a common pattern, use the detailed analysis. This always works.**

9.3.1 `for` common patterns

Common pattern	What this means	Values of `x`
`for (x = A; x < B; ++x) ZZ`	repeat `ZZ` for `x` from `A` to `B` (exclusive)	`A`, `A+1`, `A+2` ... `B-1`
`for (x = A; x <= B; ++x) ZZ`	repeat `ZZ` for `x` from `A` to `B` (inclusive)	`A`, `A+1`, `A+2` ... `B`
`for (x = A; x < B; x = x + C) ZZ`	repeat `ZZ` for `x` from `A` to `B` (exclusive) by `C` increments	`A`, `A+C`, `A+2C` ... (up to and including the last value `A+k*C` smaller than `B`)
`for (x = A; x > B; --x) ZZ`	repeat `ZZ` for `x` from `A` down to `B` (exclusive)	`A`, `A-1`, `A-2`, ... `B+1`
`for (x = A; x >= B; --x) ZZ`	repeat `ZZ` for `x` from `A` down to `B` (inclusive)	`A`, `A-1`, `A-2`, ... `B`
`for (x = A; x > B; x = x - C) ZZ`	repeat `ZZ` for `x` from `A` down to `B` (exclusive) by `C` decrements	`A`, `A-C`, `A-2C`, `A-3C` ...

For example, you've seen this before:

```
#include <stdio.h>
int main(int argc, string argv[]) {
  int i;
  for (i = 1; i < argc; ++i) {
    printf("%s\n", argv[i]);
  }
  return 0;
}
```

Here `for (i = 1; i < argc; ++i)` makes `i` go from `1` to the value of `argc` (exclusive), and executes `printf(...)` each time.

Another one:

```
#include <stdio.h>
int main() {
  int c;
  for (c = 'a'; c <= 'z'; ++c) {
    printf("%c = %d\n", c, c);
  }
  return 0;
}
```

What does this do? Remember, literal characters evaluate to the character code!

```
$ gedit test16.c # put code in file
$ cc test16.c
$ ./a.out
a 97
b 98
c 99
...
y 121
z 122
$
```

9.3.2 `for` general form and definition

General syntax:

```
for (a ; b ; c) d
```

Behavior:

1. evaluates `a`.
2. evaluates `b`.
3. if the value of `b` is non-zero, then:
    - runs `d`, then
    - evaluates `c`, then
    - start again at step 2.

That's all!

You can imagine the following substitution:

```
for ( ○ ; □ ; ▵ ) ◇
```

is equivalent to:

```
○;
while (□) {
    ◇
    ▵;
}
```

The three expressions are called the *start expression*, the *condition expression* and the *step expression*.

In other words:

```
for ( start; condition; step ) { something; }
```

is equivalent to:

```
start;
while(condition) {
   { something; }
   step;
}
```

Let's try it:

`for` code	Equivalent to

`for(i = 0; i < 10; ++i)`
`  printf("%d\n", i);`
	`i = 0;`
	`while (i < 10) {`
	`  printf("%d\n", i);`
	`  ++i;`
	`}`

`for(c = 'z'; c >= 'a'; --c)`
`  printf("%c %d\n", c, c);`
	`c = 'z';`
	`while (c >= 'a') {`
	`  printf("%c %d\n", c, c);`
	`  --c;`
	`}`

9.4 Summary

- `if (a) b else c` : "if a then b else c"
  - the parentheses are mandatory around the condition!

    "`if (x == a) { b }`", not "`if x == a { b }`"

  - `else` binds tightly!

    "`if (a) if (b) c else d`" equivalent to "`if (a) { if (b) c else d }`"

- `while` : "repeat as long as the condition is true"
- `do .. while` : "do once, then repeat as long as the condition is true"
- `for (start ; cond ; step) thing` : "do `start`, then repeat `thing` then `step` as long as `cond` is true"

10 C's increment and decrement expressions

Thanks to "`for`" you have learned about some more expressions!

Category	Format	What we call it in source code	Its value during execution
unary	`++a`	"plus plus a"	increment a by 1, then return the new value **after** the increment
unary	`--a`	"minus minus a"	decrement a by 1, then return the new value **after** the increment
unary	`a++`	"a plus plus"	the value of a **before** it is incremented by 1
unary	`a--`	"a minus minus"	the value of a **before** it is decremented by 1

What does the following program print?

```
#include <stdio.h>
int main() {
  int a = 42, b = 69;
  printf("%d %d ", a++, ++b);
  printf("%d %d\n", a, b);
  return 0
}
```

poll: progc8 poll link - poll results

11 Storing stuff: variables

You have seen plenty of variables already!

Here:

```
    int a = 40 + 2;  // a is a variable!
```

```
    for (int i = 0; i < 20; i++) // i is a variable!
        printf("%d\n", i);
```

```
int main(int argc, string argv[]) {
  // argc and argv are variables!
  ...
}
```

11.1 Variable definitions

What's common to all this:

- you *declare a new variable* using the general syntax "`type name;`"

  (the name of a type, a space, then the name of your new variable, followed by a final semicolon).

For example, "`int a;`" or "`string b;`".

- you can optionally set an initial value at the same time as the declaration.

  For example, "`int a = 42;`" or "`string b = "hello";`"

- you can declare multiple variables separated by commas:

  For example, "`int a, b;`"

  Or: "`int a = 42, b = 69;`"

- the arguments to a functions are variables too!

11.2 Variable assignments

You can set a variable to a new value:

- during its declaration, with the syntax "`type name = value;`" (see above);
- using the **assignment operator**: "`name = newvalue`" (you've seen this already!);
- using the increment/decrement operators: "`name++`", etc. (see above too!)

Beware! Do not use a variable before it is initialized.

For example, the following program is invalid:

```
int main() {
   int a;
   return a;
   a = 42;
}
```

```
$ gedit test17.c # put code in file
$ cc test17.c
test17.c: In function ‘main’:
test17.c:3:11: warning: ‘a’ is used uninitialized in this function [-Wuninitialized]
  return a;
         ^
```

We can't run this program. Really. Don't try!

Don't use uninitialized variables!

Another example invalid use:

```
#include <stdio.h>
int main() {
   int c;
   while(c != EOF) {  // poor kitten!
      c = getchar();
      putchar(c);
   }
   return 0;
}
```

12 Definitions and declarations

- *Definition*: this is a thing, and here is how it works.
- *Declaration*: I promise to the compiler that this thing exists. Somewhere else.

12.1 Function definitions

General syntax:

```
type name(args...) {
    // ... stuff ...
    return value;
    // ... stuff ...
}
```

You read this as: "the *return type*, then the *function name*, then the *parameter definitions* inside parentheses, then the *body* of the function enclosed in braces."

You can use the `return` statement at any point to stop the execution of the function and return the value of the expression to the *caller*.

12.2 Function declaration

General syntax:

```
type name(args...);

// or, optionally:
extern type name(args...);
```

You read this as "the optional keyword `extern`, then the *return type*, then the *function name*, then the *parameter declarations* inside parentheses, then a semicolon."

Notes:

- function declarations are optional: the compiler will make one automatically if you use a function that is not declared; however
- **an implicit function declaration will cause a warning** and you should consider this a programming error.
- a function definition is also a declaration.

Always declare your functions if you don't define them in the same source file!

12.3 Header files

Remember the thing "`#include <stdio.h>`"?

What does this mean?

There's no magic: **it tells the compiler to take the file `stdio.h` and copy it inside your program.**

Look at what's in stdio.h yourself:

```
extern int printf(string fmt, ...);
extern int putchar(int);
extern int getchar();
#define EOF -1
```

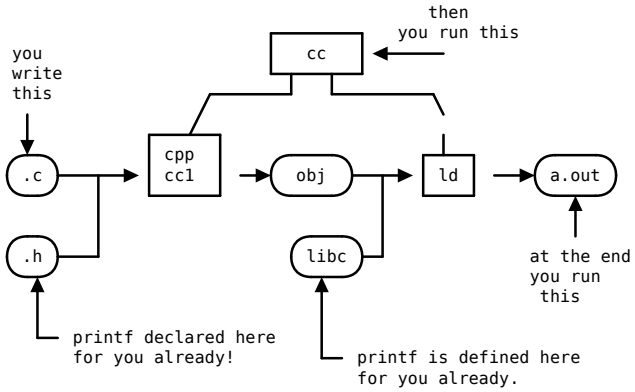
In other words we don't *really* need to use `#include <stdio.h>`, the following works just as well:

```
// Look ma', no #include!
// We just need a suitable declaration of printf.
extern int printf(string fmt, ...);

int main() {
  // just works!
  printf("hello!\n");
  return 0;
}
```

12.4 Compilation and linking

So where does `printf` really come from?



12.5 Summary

- A *definition* makes a function "here and now".
- A *declaration* promises the function exists somewhere else.
  - If you forget a function declaration, the C compiler makes one for you automatically (with a warning).
- A *header file* is just a bunch of declarations.
- `cc` is a command that really runs three other commands: `cpp`, `cc1` and `ld`.
- `cpp` groups *source code* and *header files* together (and produces source code);-
  - headers are just copy-pasted in the source code where `#include` is used! Nothing more.
- `cc1` *translates* (compiles) source code to *object code*
- `ld` groups object code together with "libc", also called *the C standard library*
  - a *library* is a collection of predefined object files, containing `printf`, `atoi`, `getchar`, etc.

13 Types

So far you have met three types:

- `int` - whole numbers
- `string` - character strings (with easy.h only!)
- `ref_of(T)` - a reference to another variable of type `T` (with easy.h only!)

13.1 Basic types

These are the common types: `int`, `char`, `double`.

Here are *all the basic types*:

Name	Synonyms	Description	Size in memory (bytes)	Possible values
`char`	N/A	A small whole number	1 <= **Sc**	either $[-2^{Sc-1} \ldots 2^{Sc-1}-1]$ or $[0 \ldots 2$ **depends on the environment**
`signed char`	N/A	A small signed whole number	1 <= **Sc**	always $[-2^{Sc-1} \ldots 2^{Sc-1}-1]$
`unsigned char`	N/A	A small unsigned (positive) whole number	1 <= **Sc**	always $[0 \ldots 2^{Sc}-1]$
`short`	`signed short`	A smallish whole number	2 <= **Ss**	$[-2^{Ss-1} \ldots 2^{Ss-1}-1]$
`unsigned short`	N/A	A smallish positive whole number	2 <= **Ss**	$[0 \ldots 2^{Ss}-1]$
`int`	`signed int`	A whole number	2 <= **Si**	$[-2^{Si-1} \ldots 2^{Si-1}-1]$
`unsigned int`	`unsigned`	A positive whole number	2 <= **Si**	$[0 \ldots 2^{Si}-1]$
`long`	`signed long`	A bigger whole number	4 <= **Sl**	$[-2^{Sl-1} \ldots 2^{Sl-1}-1]$
`unsigned long`	N/A	A bigger positive whole number	4 <= **Sl**	$[0 \ldots 2^{Sl}-1]$
`long long`	`signed long long`	A yet bigger whole number	8 <= **Sll**	$[-2^{Sll-1} \ldots 2^{Sll-1}-1]$
`unsigned long long`	N/A	A yet bigger positive whole number	8 <= **Sll**	$[0 \ldots 2^{Sll}-1]$
`float`	N/A	A single precision FP number	4	$[-3.40 \times 10^{38} \ldots 3.40 \times 10^{38}] \cup \{-$
`double`	N/A	A single precision FP number	8	$[-1.80 \times 10^{308} \ldots 1.80 \times 10^{308}] \cup \{$

Guarantee: **Sc <= Ss <= Si <= Sl <= Sll**

Some C implementations also provide "`long double`" but this is non-standard.

For more information about floating-point numbers: http://liv.science.uva.nl/dfp.html

Which type should you use in your own programs?

- if you need a whole number: use `int`, unless you have a good reason to use something else.
- if you need a floating-point number: use `double`, unless you have a good reason to use `float`.
- if you need a character: use `int`, except when you need an array of characters, then use `char`.

Side question: why does C provide so many types if one only needs `int`, `char` and `double` in most programs? That's because these options give control to the programmer over memory usage and performance. Once you start caring about memory usage and performance, you can start thinking about this again. (note: not in this course.)

13.2 Composite types

Here are *all the composite types*:

- *complex types*: if `T` is a type, then "`typedef _Complex T t`" defines a new type `t`:
  - it's the type of a complex number with real and imaginary parts of type `T`
  - its size in memory is `2 * sizeof(T)`.
- *reference types*: if `T` is a type, then "`typedef T *t`" (or "`typedef ref_to(T) t`" with easy.h) defines a new type `t`:
  - it's the type of a reference to a variable of type `T`
  - its size in memory depends on the environment: usually 2, 4 or 8 bytes.
- *void reference type*: "`void*`" is a type;
  - it's the type of a reference to a variable of unknown type;
  - its size in memory depends on the environment;
- *array types*: if `T` is a type, then "`typedef T t[N]`" or "`typedef T t[]`" defines a new type `t`:
  - it's the type of arrays of values of type `T`;
  - its size in memory is either `N * sizeof(T)` if `N` is specified, or unknown otherwise.
- *function types*: if `T`, `U`, `V` are types, then "`typedef T (*t)(U, V)`" defines a new type `t`;
  - it's the type of references to functions taking parameters of type `U` and `V` and returning values of type `T`;
  - its size in memory is the same as the size of `void*`
- all the *structure types* (see below);
- all the *union types* (see below);
- all the *enumeration types* (see below).

Here are some of the standard additional types predefined for you with `typedef` in `stdint.h`:

Name	Description	Size in memory (bytes)	Possible values
`int8_t`	A 8-bit whole number	1	$-128..127$
`uint8_t`	A 8-bit positive whole number	1	$0..255$
`int16_t`	A 16-bit whole number	2	$-32768..32767$
`uint16_t`	A 16-bit positive whole number	2	$0..65535$
`int32_t`	A 32-bit whole number	4	$-2147483648..2147483647$
`uint32_t`	A 32-bit positive whole number	4	$0..4294967295$
`int64_t`	A 64-bit whole number	8	$-9223372036854775808..9223372036854775807$
`uint64_t`	A 64-bit positive whole number	8	$0..18446744073709551615$
`intptr_t`	A whole number of size at least as large as `sizeof(void*)`	(depends)	(depends)
`uintptr_t`	A positive whole number of size at least as large as `sizeof(void*)`	(depends)	(depends)
`intmax_t`	A whole number of the maximum supported size	(depends)	(depends)
`uintmax_t`	A positive whole number of the maximum supported size	(depends)	(depends)

When are these useful? In this course, probably never. In the course "operating systems", perhaps once or twice. If you use C professionally, surely.

13.3 Array types

Array types in C are *weird*.

At first they look all right:

```
// Returns the nth element of somearray.
int pick(int n, int somearray[10]) {
  return somearray[n];
}

int myarray[10] = { 3, 2, 1, 4, 5, 7, 1, 3, 4, 5 };

int main() {
   // The function takes an array of 10 ints as parameter;
   // The call gives an array of 10 ints as argument.
   // Everything good.
   return pick(3, myarray);
}
```

Then you figure that the following is correct, too:

```
// Returns the nth element of somearray.
int pick(int n, int somearray[]) {
  return somearray[n];
}

int myarray[10] = { 3, 2, 1, 4, 5, 7, 1, 3, 4, 5 };

int main() {
   // The function takes an array of ints as parameter, size unknown;
   // The call gives an array of 10 ints as argument.
   // This works.
   return pick(3, myarray);
}
```

A function argument can have an array type of unspecified size.

An argument whose array type has a known size **automatically degenerates** to an array type of unknown size.

Or in other words:

- the **definition** of an array variable must specify its size;
- the **declaration** of an array variable can omit its size.

Perhaps not too weird yet, but now go figure this out:

```
// Returns the nth element of somearray.
int pick(int n, int somearray[]) {
  ref_to(int) r = somearray;              // WEIRD
  return r[n];                            // WOOO WEIRD
}

int myarray[10] = { 3, 2, 1, 4, 5, 7, 1, 3, 4, 5 };

int main() {
   ref_to(int) r = myarray; // WEIRD AGAIN
   return pick(3, r); // r is reference, function wants array, HOW IS THIS POSSIBLE?
}
```

Woah.

To understand this, you must absolutely grok four completely **WEIRD** separate mechanisms that happen to work together in this example.

1) **An expression of array type always automatically degenerates to a reference to its first element in every context.**

So this is valid: `int a[10];  ref_to(int) b = a;`

It defines `b` as a reference to the first element of `a`. It is equivalent to: `int a[10]; ref_to(int) b = ref(a[0]);`

This is valid too: `int a[10];  deref(a) = 42;`

It sets the first element of `a` to 42.

2) **It is always valid to give an expression with a reference type to a function that expects an array type.**

So suppose you have a function `int foo(int a[10]) { return a[5]; }`

This is valid: `int b[10];  foo(b);`

This is valid, too: `int b[10]; ref_to(int) c = b; foo(c);`

Now, this is ... dangerous to kittens: `int b[3]; foo(b);`

For ... reasons ... the C compiler *accepts to compile* this last example but *its run-time behavior is undefined* (= kittens are eaten).

3) **The expression `a[b]` is always strictly equivalent to `deref(a+b)` or `*(a+b)`.** This is a pure syntactic equivalence — there is no magic.

4) **If `a` is an expression that is a reference to an element of an array, then the expression `a+N` is a reference to the Nth element after the current element.**

That's it! It's both devilishly simple and mind-boggedly complex.

Learn these 4 rules by heart.

Now let's deconstruct the example again:

```
int pick(int n, int somearray[]) {
  // here "somearray" has an array type.

  // The following assignment is equivalent to:
  //    ref_to(int) r = ref(somearray[0]);  // rule 1
  ref_to(int) r = somearray;

  // The following statement "return r[n]" is equivalent to:
  //    ref_to(int) rn    = r + n;    // rule 3
  //    int         value = deref(rn);      // rule 4
  //    return value;
  return r[n];
}

int main() {
   // In the following call "pick(3, myarray)"
   // The expression "myarray" automatically degenerates:
   // the call is equivalent to:
   //    ref_to(int) r = ref_to(a[0]);  // rule 1
   //    pick(3, r);                    // call valid, rule 2
   return pick(3, myarray);
}
```

13.3.1 Pseudo `string` type

Of course the `string` type does not really exist in C.

In easy.h you see the following:

```
typedef char* string;
```

What's going on?

Every time you have seen "`string`" in the previous examples you were really using "`ref_to(char)`" or "`char*`".

Meanwhile, all the *string literals* (of the form `"..."` in code) have type `char[]`, i.e., array of `char`.

So we have:

```
int printf(ref_to(char), ...);

int main() {
   // The following is equivalent to:
   //    char         s[6] = { 'h', 'e', 'l', 'l', 'o', 0 };
   //    ref_to(char) p    = ref(s[0]); // rule 1
   //    printf(p);
   printf("hello");
   return 0;
}
```

13.3.2 Commutativity of addition

Remember, `a + b == b + a`?

Now, rule 3 above says `a[b]` is strictly equivalent to `deref(a + b)`

So it's also equivalent to `deref(b + a)`

So it's also equivalent to `b[a]`

Wooo! Magic!

What does this program do?

```
int main() {
  printf("hello %c!", 1["world"]);
  return 0;
}
```

13.4 Enumeration types

These are boring: the syntax "`enum { A, B, C }`" is an integer type which can only have the symbolic values `A`, `B` or `C`.

It's possible to also specify numeric values: "`enum { A = 12, B = 23, C = 34 }`"

Then it's possible to give the entire enumeration a name:

```
typedef enum { A, B, C } myenum;
```

This way one can define more variables with the new type `myenum`. For example:

```
typedef enum { A, B, C } myenum;

int main() {
   int x = A; // valid
   myenum y = B; // valid
   myenum z = 435; // warning (kittens are eaten)

   return x + y + z; // valid: they are just numbers.
}
```

The names used inside the `enum` declaration are called *enumeration constants*.

13.5 Aggregate types: structures

A *structure type* allows you to **logically group multiple variables together.**

Think about a "class" in Java or Python to do something similar.

Why does this matter? Often an application will represent one thing using multiple variable. For example a person can have a name, an age, a gender, an address. If you have more than one persons, you need more variables. Structure types can help organize the code and make it more readable.

So how does this work?

13.5.1 Structure definitions

You define a new structure type like this: `struct { fields... }`

Some examples:

```c
// This defines an anonymous struct. Not very useful.
struct {
  char*   name;
  int     age;
  char*   address;
};

// This defines a struct with a name.
struct person {
  char*   name;
  int     age;
  char*   address;
};

// This creates two "objects" with type "struct person".
// The word "struct" is mandatory.
struct person joe = { "joe", 34, "33 elm street" };
struct person jane = { "jane", 23, "1 circle lane" };

// This gives a short name to the struct.
typedef struct person person_t;

// Now we can use the short name.
person_t mark = { "mark", 18, "1 circle lane" };
person_t penelope = { "penelope", 44, "123 61st st"};
```

What's going on?

- Inside "`struct { ... }`" you declare the *members* of the struct.
- The name after the word "`struct`" becomes the name of the struct. That name must always be used in combination with "`struct`", like "`struct person`", "`struct stack`", etc.
- You can also make a short name with: `typedef struct structname shortname;`

Some oddities:

- You can define a short name without a struct name: `typedef struct { ... } shortname;`
- You can define a struct at the same time as one or more variables of its type:
  `struct person { ... } joe, jane, mark, penelope;`

13.5.2 Accessing a member of a struct in expressions

For example:

```c
// Define a struct to represent a person.
struct person {
  char*   name;
  int     age;
  char*   address;
};
// Make a short name.
typedef struct person person_t;

int age_difference(person_t someone, person_t someone_else) {

   // New syntax: x.y
   int diff = someone.age - someone_else.age;

   if (diff < 0)
      diff = -diff;
   return diff;
}
```

General syntax: **if x is an object of struct type, then the expression `x.a` accesses the member a of x.**

Another example:

```c
int main() {
  person_t joe;
  joe.name = "joe";
  joe.age = 34;
  joe.address = "33 elm street";
  printf("name: %s, age: %d, addr: %s\n", joe.name, joe.age, joe.address);
  return 0;
}
```

13.5.3 Accessing a member of a struct via a reference

Say you want to make a function that updates a person record to indicate that person has moved to a new address. The following works (with easy.h):

```
void move_to(ref_to(person_t) someone, string new_address) {
   printf("person %s moving from %s to %s\n",
      deref(someone).name,
      deref(someone).address,
      new_address);
   deref(someone).address = new_address;
}
```

Now, it just so happens that this kind of code is **very common**. So common, in fact, that the designers of C found it was a good idea to provide a *shorter syntax* to write **equivalent but shorter code**:

```
void move_to(ref_to(person_t) someone, string new_address) {
   printf("person %s moving from %s to %s\n",
      someone->name,
      someone->address,
      new_address);
   someone->address = new_address;
}
```

General definition:

- **if `x` is a reference to a struct, then the expression `x->a` accesses the member `a` of the referenced struct.**
- (More general): **the syntax "`a->b`" is always strictly equivalent to "`deref(a).b`" (or "`(*a).b`").**

13.5.4 Size of a struct in memory

The size of a struct (e.g. `struct person`) can be computed as usual with `sizeof`: `sizeof(struct person)`

Can you predict the size of a struct? Usually, but not always accurately. Rule of thumb: **the size of a struct is the sum of the sizes of its members, plus an epsilon.**

(It is not exactly the sum of the size of the members, because of alignment issues. Not important for this course.)

13.6 Aggregate types: unions

The idea of *union types* in C is an example of an *atavism*: a primitive feature that has become rather useless or inconvenient but not yet weeded out by evolution.

Unfortunately you have to understand them, and even more unfortunately perhaps use them, but know that virtually every language since C has done them better. Eventually you will realize that C's unions are a degenerate, under-equipped ancestor of the much more useful, widely acclaimed sum types, also called tagged union or variant in some languages.

So, where does this leave us?

General rule: **the C compiler handles `union` more or less in the same way as `struct`, except that all the members overlap in memory.**

In effect:

- to define a `union` use the same syntax as `struct`, just replace the keyword:
  - anonymous union: `union { int a; int b; }`
  - named union: `union blah { int a; int b; }`
  - short name: `typedef union blah blih;`
- make variables again by replacing the keyword: `union blah myvar;`
- access members: `myvar.a`
- access members over a reference: `deref(myvar).a` or `myvar->a`

What's the difference then?

1. **all the members of an union object start at the same place in memory.**
2. **the size of an union is the size of its largest member.**

When do you need to use a `union`? Not applicable in this course (I think).

14 Next topics

- references, pointers and address computations (generalized arrays)
- linkage, storage and lifetimes
- declarators